Laravel Video API

Role-Based Access: Student & Staff

A practical guide to structuring a Laravel API with **student** and **staff** roles using Middleware, Policies, and a single clean Controller.

01 — The Scenario

Role	Allowed Methods	Description
`Student`	index, show	Read-only — can only watch videos
`Staff`	index, show, store, update, destroy	Full CRUD — owns uploads

02 — Migration

Create the **videos** table. The user_id foreign key tracks which staff member uploaded each video.

```
Schema::create('videos' , function (Blueprint $table) {

$table-&gt;id();

$table-&gt;foreignId('user_id' )-&gt;constrained()-&gt;cascadeOnDelete();

$table-&gt;string('title' );

$table-&gt;string('description' )-&gt;nullable();

$table-&gt;string('file_path' );

$table-&gt;timestamps();

});
```

03 — Routes (routes/api.php)

Middleware enforces **role access at the route level** — controllers never need to check roles manually.

```
Route::middleware('auth:sanctum' )-&gt;group(function () {
```

```
// Student — read only

Route::middleware('role:student' )-&gt;group(function () {

Route::get('videos' , [VideoController::class, 'index' ]);

Route::get('videos/{video}' , [VideoController::class, 'show' ]);

});


// Staff — full control

Route::middleware('role:staff' )-&gt;group(function () {

Route::get('videos' , [VideoController::class, 'index' ]);

Route::get('videos/{video}' , [VideoController::class, 'show' ]);

Route::post('videos' , [VideoController::class, 'store' ]);

Route::put('videos/{video}' , [VideoController::class, 'update' ]);

Route::delete('videos/{video}' , [VideoController::class, 'destroy' ]);

});

});
```

04 — Policy (app/Policies/VideoPolicy.php)

The **before()** hook lets staff bypass all checks. Individual methods define what students are explicitly allowed or denied.

```
class VideoPolicy

{

// Staff bypasses every policy check

public function before(User $user, string $ability): bool|null

{

if ($user-&gt;role === 'staff' ) return true;
```

```
return null; // fall through for students

}


// Any authenticated student may watch

public function view(User $user, Video $video): bool

{

return true;

}


// Students can never edit or delete

public function update(User $user, Video $video): bool

{

return false;

}


public function delete(User $user, Video $video): bool

{

return false;

}

}
```

05 — Controller (app/Http/Controllers/VideoController.php)

A **single controller** serves both roles. Middleware blocks students from write routes before the request even reaches these methods.

```
class VideoController extends Controller

{
```

```
// Both student and staff

public function index()

{

return VideoResource::collection(Video::with('user' )-&gt;paginate());

}


// Both student and staff

public function show(Video $video)

{

return new VideoResource($video);

}


// Staff only (middleware blocks students)

public function store(StoreVideoRequest $request)

{

$video = auth()-&gt;user()-&gt;videos()-&gt;create($request-&gt;validated());

return new VideoResource($video);

}


// Staff only

public function update(UpdateVideoRequest $request, Video $video)

{

$this-&gt;authorize('update' , $video);

$video-&gt;update($request-&gt;validated());
```

```
return new VideoResource($video);

}


// Staff only

public function destroy(Video $video)

{

$this-&gt;authorize('delete' , $video);

$video-&gt;delete();

return response()-&gt;noContent();

}

}
```

06 — Layer Summary

Component	Layer	Responsibility
`auth:sanctum`	Middleware	Blocks unauthenticated requests
`role:student/staff`	Middleware	Blocks wrong role from route group
`before()`	Policy	Staff bypasses all ownership checks
`view/update/delete`	Policy	Defines what students can/cannot do
`Controller`	Controller	Pure business logic — no role conditionals

Split by **behavior**, not by who. If logic genuinely diverges between roles, separate controllers earn their place — otherwise one controller + middleware + policy is the cleaner standard.